

Quick Basket Checkout App

Automating Small Business Sales with Python

Project Workflow Document

1. Project Overview

Quick Basket is a lightweight, offline-friendly checkout application built in Python for small retailers — provisions stores, boutiques, pharmacies, and similar businesses across Africa's retail ecosystem. It runs entirely on a personal computer with no internet connection or database server required, making it practical for shops where connectivity is unreliable or costly.

The application allows a cashier to record items sold, automatically calculates the subtotal, applies an optional discount and standard VAT, prints a clear itemised receipt, and saves that receipt as an image file that can be shared over WhatsApp, printed, or archived for record-keeping.

This document walks through the five stages of the build, explains the reasoning behind each design decision, and shows the diagrams and sample output produced along the way. The accompanying Jupyter notebook (QuickBasket_Checkout_App.ipynb) contains the full, executable, beginner-annotated code.

2. End-to-End Workflow

The build follows five sequential stages, each producing the inputs the next stage needs:

Quick Basket Checkout App — Project Workflow

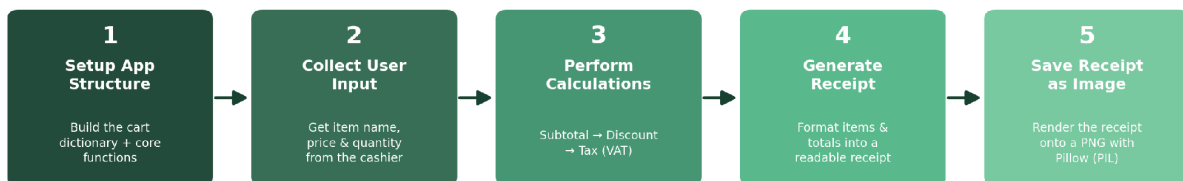


Figure 1. The five-stage Quick Basket workflow, from app structure to a shareable receipt image.

3. Step 1 — Setup App Structure

Every checkout system needs a way to remember what's currently in a customer's basket. Quick Basket represents this using a Python dictionary called `cart`, where each key is an item name and each value is a nested dictionary holding that item's price and quantity.

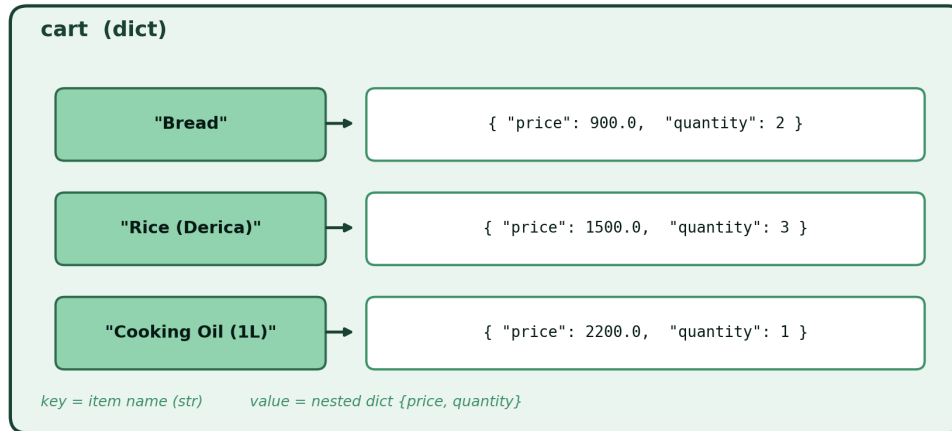


Figure 2. The nested-dictionary shape of the cart data structure.

Two small functions manage this structure so the rest of the program never touches the dictionary directly:

- `add_item(cart, name, price, quantity)` — adds a new item, or increases the quantity if it already exists.
- `remove_item(cart, name)` — deletes an item from the cart entirely.

Sample from the notebook:

```
cart = {}

def add_item(cart, name, price, quantity):
    name = name.strip().title()
    if name in cart:
        cart[name]["quantity"] += quantity
    else:
        cart[name] = {"price": price, "quantity": quantity}
```

4. Step 2 — Collect User Input

The cashier enters each item's name, price, and quantity. Python's built-in `input()` function is used for this, wrapped in a loop that keeps asking for items until the cashier types "done". A `try/except` block guards against typing errors (e.g. entering letters where a price is expected), so the app doesn't crash mid-sale.

```
while True:
    name = input("Item name: ").strip()
    if name.lower() == "done":
        break
    try:
        price = float(input(f"Price of '{name}' (Naira): "))
        quantity = int(input(f"Quantity of '{name}': "))
    except ValueError:
        print("Please enter numeric values. Try again.")
        continue
```

```
add_item(cart, name, price, quantity)
```

For automated or demo runs (such as the executed notebook), a `load_demo_items()` helper populates the cart from a ready-made list, exercising the same `add_item()` logic without requiring manual typing.

5. Step 3 — Perform Calculations

Pricing is split into small, single-purpose functions so each rule can be tested and reused independently:

- `calculate_subtotal(cart)` — sums $\text{price} \times \text{quantity}$ across every item.
- `apply_discount(amount, discount_percent)` — removes an optional percentage-based discount.
- `apply_tax(amount, tax_percent)` — adds VAT on top (Nigeria's standard rate of 7.5% is the default).
- `calculate_totals(cart, discount_percent, tax_percent)` — runs `subtotal` → `discount` → `tax` and returns every figure the receipt needs.

```
def calculate_totals(cart, discount_percent=0, tax_percent=7.5):
    subtotal = calculate_subtotal(cart)
    after_discount, discount_amount = apply_discount(subtotal, discount_percent)
    final_total, tax_amount = apply_tax(after_discount, tax_percent)
    return {
        "subtotal": subtotal, "discount_amount": discount_amount,
        "tax_amount": tax_amount, "final_total": final_total,
        "discount_percent": discount_percent, "tax_percent": tax_percent,
    }
```

6. Step 4 — Generate Receipt

`generate_receipt()` builds the receipt as plain text, line by line, using f-strings and alignment operators (`<`20 for left-aligned columns, `>`10 for right-aligned figures) so item names, quantities, unit prices and line totals line up visually, the same way a till receipt does.

```
lines.append(f"{'Item':<20}{'Qty':>4}{'Price':>8}{'Total':>10}")
for name, details in cart.items():
    item_total = details["price"] * details["quantity"]
    lines.append(
        f"{'name':<20}{'details['quantity']':>4}"
        f"{'details['price']':>8,.0f}{'item_total':>10,.0f}"
    )
return "\n".join(lines)
```

The result is a single string, ready to `print()` to the screen, log to a file, or hand to Step 5 for image export. The final printed receipt from a sample sale looks like this:

7. Step 5 — Save the Receipt as an Image

To make the receipt shareable outside the terminal, `save_receipt_as_image()` uses the Pillow (PIL) library to draw the receipt text onto a blank white canvas with a monospaced font, then saves it as a PNG file — ready to send on WhatsApp, print, or store for accounting records.

Mama Ngozi's Provisions
Lagos, Nigeria

Date: 11-09-2026 22:11
Cashier: Tolu A.

Item	Qty	Price	Total
Rice (Derica)	3	1,500	4,500
Bread	2	900	1,800
Cooking Oil (1L)	1	2,200	2,200
Beans (Derica)	4	1,200	4,800
Sugar (500G)	2	800	1,600

Subtotal: 14,900.00
Discount (5%): - 745.00
VAT (7.5%): 1,061.62
=====

TOTAL DUE (N): 15,216.62
=====

Thank you for shopping with us!
Come back soon!

Figure 3. Sample receipt generated and saved as an image by the notebook.

```
font = ImageFont.truetype(
    "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", 16)
img = Image.new("RGB", (canvas_width, canvas_height), color="white")
draw = ImageDraw.Draw(img)
for line in receipt_text.split("\n"):
    draw.text((padding, y), line, fill="black", font=font)
    y += line_height
img.save(filename)
```

8. Python Concepts Covered

Step / Function	Core Python Concept
cart dictionary, add_item(), remove_item()	Dictionaries (incl. nested), functions, mutability
collect_items_interactively()	input(), while loops, try/except, type conversion
calculate_subtotal(), apply_discount(), apply_tax()	Functions, default arguments, generator expressions
generate_receipt()	f-strings, string formatting and alignment
save_receipt_as_image()	Working with an external library (Pillow / PIL)

9. Ideas for Extending the Project

- Persist sales history to a CSV file or a small SQLite database for daily reconciliation.
- Add a product lookup dictionary so cashiers select items by code instead of typing full names.

- Track payment method (cash, card, transfer) alongside each sale.
- Wrap the existing functions in a tkinter GUI so cashiers click instead of typing in a terminal.
- Parameterise the currency symbol to support retailers outside Nigeria.

10. Summary

Quick Basket demonstrates how a handful of small, well-named Python functions — covering data structures, input handling, arithmetic, string formatting, and an external imaging library — can be combined into a genuinely useful, offline tool for small retailers. Each stage is intentionally decoupled, so any single piece (input method, pricing rules, receipt layout, or export format) can be upgraded independently as the business's needs grow.